

Agentic AI Phishing Detector

Technical Documentation & System Architecture

Dual-Engine MCO Architecture for Automated Phishing Detection and Prevention

Based on: "**Agentic AI for Phishing Detection and Prevention**"

Loo, Galindo, Romero et al. - Universidad Tecnologica de Honduras (UTH), 2025

DistilBERT NLP	Random Forest	Agentic AI	Explainable	Bilingual (EN/ES)
----------------	---------------	------------	-------------	-------------------

Table of Contents

1. Architecture Overview
2. Phase 1: Monitoring (Input Parsing)
3. Phase 2: Classification (Dual-Engine Analysis)
 - 3.1 Engine A - Semantic Understanding
 - 3.2 Engine B - Structural Analysis
4. Feature Fusion & Classification
5. Explainability Layer
6. Phase 3: Optimization (Training Pipeline)
 - 6.1 Training Data: SpaPhish Dataset
 - 6.2 Training Process & Results
7. Security Advisor (Claude API)
8. Data Persistence & Dashboard
9. Technology Stack

1. Architecture Overview

The system follows a three-phase agentic loop: **Monitoring** (input parsing and normalization), **Classification** (dual-engine semantic + structural analysis fused into a Random Forest classifier), and **Optimization** (retraining with labeled samples for continuous improvement).

Phase	Component	Description
MONITORING	Input Parsers	Parse .eml files, raw text, or screenshots (OCR) into normalized email dict
CLASSIFICATION	Engine A: Semantic	DistilBERT embedding (768-d) + rule-based NLP scoring (urgency, authority, pressure, c
CLASSIFICATION	Engine B: Structural	URL analysis + typosquatting, SPF/DKIM/DMARC headers, HTML hidden elements (~50
CLASSIFICATION	Feature Fusion	Concatenate 20-d semantic + 50-d structural = 70-d feature vector
CLASSIFICATION	Classifier	Random Forest (100 trees) or heuristic fallback with multi-signal boost
OPTIMIZATION	Explainer	Human-readable explanations, education notes, threat indicator breakdown
OPTIMIZATION	Retraining	Label samples via API, retrain RF, persist to rf_classifier.pkl

Data flow: Email Input --> Parsers --> Engine A + Engine B (parallel) --> Feature Fusion --> Random Forest / Heuristic Classifier --> Verdict + Confidence + Action + Explanation --> SQLite DB (scan history) + Dashboard + Security Advisor chatbot.

2. Phase 1: Monitoring (Input Parsing)

Three input parsers normalize any email format into a standardized dictionary containing subject, body_text, body_html, and headers (if available from .eml).

Parser	Input	Method
EML Parser	.eml file bytes	Python email.message_from_bytes() with policy.default. Walks multipart MIME, extracts Subject, From
Text Parser	Raw subject + body	Detects embedded HTML via regex, strips tags. Returns standardized dict matching EML parser output
OCR Parser	PNG/JPG screenshot	Tesseract image_to_string(), computes confidence from image_to_data(), heuristically extracts subject

3. Phase 2: Classification (Dual-Engine Analysis)

3.1 Engine A - Semantic Understanding

Detects linguistic patterns indicative of social engineering attacks. Two paths run in parallel:

Path 1: DistilBERT Transformer (optional)

Model: distilbert-base-uncased (66M parameters). Tokenizes with max_length=512, extracts [CLS] token embedding producing a **768-dimensional** dense vector capturing deep semantic relationships.

Path 2: Rule-Based Pattern Scoring (always runs)

Category	# Patterns	Language	Examples
Urgency	~90	EN + ES	"account suspended", "cuenta cerrada", "verify now", "se cerrara su cuenta"
Authority	~70	EN + ES	"paypal", "microsoft", "administrador", "office 365"
Pressure	~55	EN + ES	"within 24 hours", "final notice", "sera eliminada"
Credential Requests	~65	EN + ES	"enter your password", "haga clic aqui", "verify your account"
Generic Greetings	~27	EN + ES	"dear customer", "estimado usuario"
Reward Lures	~20	EN	"congratulations", "you have won", "claim your refund"
Grammar Anomalies	Heuristic	Any	Excessive caps (>30%), punctuation (!!!), known misspellings
Brand Mentions	30+	Any	PayPal, Microsoft, Amazon, Netflix, Chase, FedEx, etc.

Scoring per category: 0 matches = 0.0, 1 = 0.35, 2 = 0.60, 3 = 0.80, 4+ = 0.90+

Combined semantic score - weighted average with three boost mechanisms:

Boost	Condition	Effect
Single-indicator	Top category score >= 0.35	Floor at score x 0.75
Multi-category	2+ categories >= 0.3	Floor at avg(top2) x 0.85
Brand co-occurrence	Brand + urgency OR credentials	Floor 0.45
	Brand + urgency + credentials	Floor 0.60

3.2 Engine B - Structural Analysis

Three sub-analyzers extract approximately 50 technical features from the email structure:

URL Analyzer (15 features)

Check	Description	Risk Score
IP-based URL	URL uses numeric IP instead of domain name	+0.25
URL Shortener	18 known shortener domains (bit.ly, t.co, etc.)	+0.15
Suspicious TLD	24 TLDs common in phishing (.xyz, .top, .buzz, etc.)	+0.15
Typosquatting	Levenshtein distance <= 2 against 45+ brand domains	+0.25
Free Hosting	30+ platforms (weebly.com, wix.com, wordpress.com, etc.)	+0.30
@ Symbol	@ in URL domain section (credential theft technique)	+0.20
Excessive Subdomains	More than 3 subdomain levels	+0.10
Suspicious Path	Keywords: login, verify, password, account (EN + ES)	+0.10
No HTTPS	URL uses HTTP instead of HTTPS	+0.05

Header Analyzer (15 features)

Check	Description	Risk Score
SPF Fail	SPF authentication check failed	+0.20
DKIM Fail	DKIM signature verification failed	+0.20
DMARC Fail	DMARC policy check failed	+0.15
Brand Mismatch	Display name contains brand but domain doesn't match (50 brands from JSON)	+0.30
Reply-To Mismatch	Reply-To domain differs from From domain	+0.20
Suspicious X-Mailer	phpmailer, swiftmailer, mass/bulk mailer	+0.05
Missing Headers	Missing Message-ID or Date headers	+0.05 each
Suspicious Chain	Received hop count > 8 or = 0	+0.10

HTML Analyzer (15 features)

Check	Description	Risk Score
Hidden Elements	7 CSS patterns: display:none, visibility:hidden, opacity:0, etc.	+0.15
External Forms	form action pointing to external domain	+0.30
Tracking Pixels	1x1 images used for tracking	+0.10
Obfuscated JS	eval(), document.write, fromCharCode, atob, hex/unicode escapes	+0.25
Iframes	Embedded iframe elements	+0.15
Base64 Content	Encoded content blocks (>2 occurrences)	+0.10

Combined structural score: URL 40% + Header 35% + HTML 25%

4. Feature Fusion & Classification

The two engines produce a **70-dimensional feature vector**:

Source	Dimension	Content
Semantic (rule-based)	20 features	Score + count for each of 6 categories, grammar score, combined score, brand mention count, p
Structural	50 features	15 URL + 15 header + 15 HTML + 5 cross-engine interaction features
Total	70 features	Concatenated into a single vector for classification

Classification Methods

Method	When Used	How It Works
Random Forest	When models/rf_classifier.pkl exists (after training)	100 estimators, max_depth=20, balanced class weights. Uses predict_proba() for
Heuristic Fallback	No trained model available	Weighted: semantic 35% + URL 30% + header 20% + HTML 15%. Multi-signal b

Decision Thresholds & Action Mapping

Confidence	Verdict	Action
>= 0.40	PHISHING	Quarantine (if > 75%) or Alert
>= 0.22	SUSPICIOUS	Alert - flag for manual review
< 0.22	LEGITIMATE	Pass - deliver normally

5. Explainability Layer

Every classification includes three human-readable outputs designed to make the system's decisions transparent and educational:

Output	Description
Explanation	Natural language paragraph citing specific indicators detected (e.g., urgency language, authority impersonation, suspicious
Education Note	Attack-type-specific teaching content: urgency tactics, credential harvesting, typosquatting, brand spoofing. Multiple notes.
Threat Indicators	5-category breakdown with scores and matched details: - Suspicious URLs (score + flagged URLs) - Urgency/Authority Language (weighted: urgency 40% + authority 30% + pressure 30%) - Header Anomalies (SPF/DKIM/DMARC + mismatch details) - Grammatical Anomalies (caps ratio, misspellings) - HTML Suspicious (hidden elements, external forms)

6. Phase 3: Optimization (Training Pipeline)

6.1 Training Data: SpaPhish Dataset

The model was trained on the **SpaPhish Dataset (DiB)**, a curated corpus of Spanish-language emails designed for phishing research:

Metric	Value
Total Samples	1,395
Phishing (label=1)	731 (52.4%)
Legitimate (label=0)	664 (47.6%)
Language	Spanish
Source Columns	hash, subject, body, date, url_count, urls, attachments_count, hops_count, Label

6.2 Training Process & Results

The training process runs each sample through the full analysis pipeline to generate feature vectors:

1. **Load CSV:** Parse all 1,395 rows from the semicolon-delimited SpaPhish dataset
2. **For each sample:** Run through semantic + structural engines:
 - parse_text_input(subject, body) --> normalized email dict
 - analyze_semantics(subject, body) --> rule scores + combined score
 - analyze_structure(parsed) --> URL/header/HTML scores + 50-feature vector
 - _rule_scores_to_features(rule_scores) --> 20-feature semantic vector
3. **Concatenate:** 20 semantic + 50 structural = 70-feature vector per sample
4. **Train RF:** RandomForestClassifier(n_estimators=100, max_depth=20, class_weight='balanced')
5. **Cross-validate:** 5-fold CV with F1 scoring
6. **Serialize:** Save to models/rf_classifier.pkl

Metric	Value
Cross-validation F1 (mean)	0.8574
Cross-validation F1 (std)	0.0207
Feature Count	70
Model File	models/rf_classifier.pkl
Classifier	RandomForestClassifier (scikit-learn)
Estimators	100
Max Depth	20
Class Weight	balanced

7. Security Advisor (Claude API)

A right-side panel chatbot powered by **Claude Sonnet** (Anthropic API) that provides proactive educational feedback after each analysis:

- **Proactive Analysis:** After each scan, automatically sends analysis context to Claude for educational feedback
- **Verdict Summary:** Posts local system messages with verdict, confidence, and key triggers
- **Educational Response:** Explains what specific indicators mean in simple terms and teaches users how to spot similar attacks
- **Interactive Q&A:** Responds to freeform security questions with full analysis context
- **Model:** claude-sonnet-4-20250514 with max_tokens=1024
- **Authentication:** User provides their own Anthropic API key (stored in browser localStorage only)

8. Data Persistence & Dashboard

SQLite database (data/phishing_detector.db) provides persistent storage across server restarts:

Table	Columns	Purpose
scan_history	id, timestamp, input_type, subject, body_preview, verdict, confidence, recency_score, explanation	Every analysis result is persisted for dashboard and history
training_samples	id, timestamp, subject, body_preview, label (0/1), features (JSON)	Labeled samples for Random Forest retraining

The **Dashboard** displays: total emails analyzed, phishing/suspicious/legitimate counts, phishing detection rate, average confidence, average threat indicator scores, and a recent analyses table. The **History** page provides paginated scan logs with filtering and full detail view.

9. Technology Stack

Layer	Technology	Purpose
Backend	Python 3.10+ / FastAPI / Uvicorn	REST API web framework and ASGI server
NLP Model	DistilBERT (HuggingFace Transformers)	768-dim semantic embeddings (optional)
Deep Learning	PyTorch	Powers DistilBERT inference (optional)
ML Classifier	scikit-learn RandomForestClassifier	100 estimators, balanced class weights
Numerical	NumPy	Feature vector operations
Email Parsing	Python email stdlib	.eml file parsing with policy.default
URL Analysis	tlldextract + python-Levenshtein	Domain extraction and typosquatting detection
OCR	Tesseract-OCR + pytesseract + Pillow	Screenshot text extraction
AI Chatbot	Anthropic Claude API	Security advisor (claude-sonnet-4-20250514)
Database	SQLite	Persistent scan history and training samples
Frontend	Tailwind CSS (CDN) + Vanilla JavaScript	Single-page application (~1290 lines)
Data Validation	Pydantic	Request/response model validation

API Endpoints

Method	Path	Description
GET	/	Serve frontend SPA
GET	/api/health	Health check
POST	/api/analyze/eml	Upload .eml file for analysis
POST	/api/analyze/text	Analyze raw text (JSON: subject + body)
POST	/api/analyze/screenshot	Upload screenshot for OCR + analysis
GET	/api/dashboard/stats	KPI statistics from database
GET	/api/scans	Paginated scan history
GET	/api/scans/{id}	Single scan detail
POST	/api/chat	Claude AI security advisor
POST	/api/train/label	Submit labeled sample for training
POST	/api/train/run	Train Random Forest on accumulated samples
GET	/api/train/stats	Training data statistics
